

DevNexus | AgileBridge

TESTING POLICY

Prepared By
DevNexus



devnexus28@gmail.com


agilebridge
CONSIDERED
SOFTWARE SOLUTIONS

Testing Policy — UniTrade

This document sets out UniTrade's testing objectives, testing types, tools and environments, defect management, acceptance criteria, and the roles of the developers responsible for testing. It reflects the system as currently built.

Testing objectives

The guiding principle is that every behavior is tested at the level where it can actually fail, and each layer has a distinct objective.

Unit tests confirm that the logic is correct in isolation — state-machine transitions, validation, guards, and calculations. They mock the repository and never touch the database.

Integration tests confirm the correctness that only holds when components interact for real rather than in isolation. On the backend this means database constraints, transactional atomicity, concurrency, query results, and the HTTP boundary, exercised against a real PostgreSQL container rather than an in-memory substitute. On the frontend it means the component tree, routing, and data-fetching working together, with the network mocked at the boundary rather than a live backend.

End-to-end tests confirm that a complete user journey works through the running application — frontend, backend, and database together — as a real user would experience it.

Some code is deliberately left untested: code that makes no decision and carries no cross-component contract, such as a controller passing a service result straight to *Ok(...)*.

Testing effort is prioritised by risk. The critical parts, tested first and more thoroughly, are the reservation lifecycle, the payment and PIN flow, chat, authentication and verification, and the background timers — the places where a defect costs a user money or access. The overall aim is confidence on these paths and protection against regressions.

Testing types

Backend and frontend each run a unit, integration, and end-to-end layer, plus two cross-cutting static checks.

Backend unit and integration tests use **xUnit** with **Coverlet**. Every test is tagged with a category trait — *[Trait("Category", "Unit")]* or *[Trait("Category", "Integration")]* — and CI runs

the two sets in separate steps that filter on that trait. The consequence is that a test with no category trait runs in neither step and is silently skipped so adding the trait is part of writing the test and reviewers check for it. Integration tests are grouped by the kind of faults they catch — flows, constraints, concurrency, access and infrastructure — rather than by production layer, so real-interaction risks stay visible at a glance.

Frontend tests use **Vitest** with the **v8** coverage provider, split into three layers by directory and dedicated **npm** scripts. Unit and Integration specs run via *test:unit* and *test:integration*; end-to-end specs live under *src/tests/e2e*, which *vite.config.ts* excludes from the Vitest run so they execute only under **Playwright**. A unit test exercises a component or hook in isolation; an integration test renders the real component tree with routing and data-fetching in place but mocks the network at the HTTP boundary (via MSW) and stubs browser-only services such as push and realtime connection.

End-to-end tests run in **Playwright** against the full-stack — database, backend, and frontend brought up together with Docker Compose. They run on pull requests into *dev* and *main* so they gate merges into the shared branches rather than running only on a nightly schedule.

Two static checks run across both stacks: **SonarCloud** for code-quality, and coverage analysis, and Gitleaks for committed secrets, which runs before anything else in each pipeline.

Tools and environments

Backend uses xUnit and Coverlet; frontend uses Vitest and v8; end-to-end uses Playwright; analysis is SonarCloud; secrets scanning is Gitleaks; dependency updates come through Dependabot. Everything runs on GitHub Actions.

Backend integration tests run against a real PostgreSQL 18 service container created fresh for each CI run. The shared integration fixture that seeds and provides that database is built before the tests that depend on it. Frontend integration tests need no such container — they run under Vitest with the network mocked via MSW. The end-to-end job brings up the database, backend, and frontend together with Docker Compose.

Defect management

Defects are tracked in GitHub issues with a severity scale — critical (money, access or data at risk), major (a critical path broken but with a workaround), minor (non-blocking), and trivial. The governing rule is that bug-fix pull requests ship with a regression test written at the level where the bug actually lived: a unit test for a logic error, an integration test for a constraint or interaction error. A fix without a regression test is not considered complete.

Acceptance criteria

A change is accepted into *dev* or *main* when:

- all unit tests and integration tests for the affected code pass, with backend integration tests run against the real PostgreSQL 18 container;
- the end-to-end suite passes against the full stack;
- no secrets are detected and static analysis (SonarCloud) reports no blocking issues;
- any new behaviors are covered by tests at the appropriate level, and any bug fix includes a regression test;
- the change has the required approvals — two for *dev*, three for *main*.

Roles and responsibilities

Each engineer owns the unit and integration tests for the module they build — the two backend engineers own their respective module suites and the frontend engineers own the frontend layers. Beyond those module owners, only two people work across module boundaries; the integration engineer, who owns the shared integration fixture and the cross-module flow tests, and the project lead.